

Connected Vehicle Intelligence Hackathon

Problem Statement & Case Study

Industry Reference: Motorq – Connected Vehicle Intelligence

Domain: Connected Vehicles · IoT · Big Data · Enterprise Architecture

Format: Open-ended build challenge

Submission: Solution Document (template provided), repository and demo video

Issued by: Talenciaglobal

Table of Contents

- [1. Introduction](#)
- [2. Industry Context: Motorq](#)
 - [2.1 Company Snapshot](#)
 - [2.2 Which Industry Motorq Belongs To](#)
 - [2.3 Business Model and Customers](#)
- [3. Industry Trends \(2026 → 2030\)](#)
 - [3.1 How Motorq Is Responding \(From Public Sources\)](#)
- [4. The Problem at Scale](#)
 - [4.1 Why a Single SQL Database Fails Here](#)
- [5. Opportunity and Problem Spaces](#)
- [6. Your Challenge \(Open-Ended\)](#)
- [7. Technical Expectations](#)
- [8. Data Engineering and Database Design](#)
- [9. Algorithms and Data Structures](#)
- [10. System Design Expectations](#)
- [11. Non-Functional Requirements](#)
- [12. Testing Expectations](#)
- [13. Deliverables](#)
- [14. Rules and Guidelines](#)
- [15. Glossary](#)
- [16. References](#)

1. Introduction

By 2030, a connected car is expected to generate up to 25 GB of data every hour. Across a fleet of 100,000 vehicles, that is a continuous firehose of location, speed, battery, diagnostic and driver-behaviour signals that must be collected, cleaned, stored, analysed and turned into decisions within seconds.

This hackathon asks your team to build an enterprise-grade solution for that world. The case study uses Motorq, a US connected-vehicle intelligence company with engineering hubs in Chennai and Bangalore, as its industry reference. You will not copy Motorq's product; you will identify a real problem in the connected-vehicle space and engineer your own solution to it.

What makes this challenge different: it is open-ended in what you build, but strict in how well you build it. Scale, data engineering, system design, security, testing and DevOps are all expected at production quality.

2. Industry Context: Motorq

2.1 Company Snapshot

Attribute	Detail
Founded	2016, San Francisco, USA
Stage	Series B; more than USD 50 million raised (Insight Partners, FM Capital, Story Ventures)
Hubs	San Francisco, Seattle, Chennai, Bangalore
Positioning	The enterprise platform for Connected Vehicle Intelligence
Scale	25+ automotive brands, 500K+ connected vehicles daily, 250B+ vehicle signals processed per month
OEM partners	Volvo Cars, Subaru of America, Stellantis (via Mobilisights, 2026 OEM Partner of the Year) and others

2.2 Which Industry Motorq Belongs To

Motorq sits where four industries meet. Understanding this helps you frame both the users and the technical constraints of your solution.

- **Automotive Technology:** works directly with vehicle manufacturers (OEMs) and their embedded telematics
- **Mobility SaaS / Enterprise Software:** sells subscriptions, portals and APIs to fleets, dealers and lenders
- **IoT and Big Data:** every vehicle is an edge device streaming high-volume, time-series data

- **Electric Vehicle Ecosystem:** supports mixed fleets of petrol, diesel, hybrid and fully electric vehicles

2.3 Business Model and Customers

Motorq's platform is device-free: instead of installing aftermarket OBD-II dongles, it pulls data directly from OEM clouds through commercial and technical partnerships. The data is normalised into one standard format and delivered as insights and workflows.

Customer Segment	Typical Use of Vehicle Data
Commercial and rental fleets	Live location, utilisation, maintenance, fuel and idling cost, driver safety
Fleet management companies	Migrating customers from legacy hardware to OEM connectivity
Lenders	Vehicle recovery, collateral monitoring (Motorq reports a 68% reduction in recovery time)
Dealerships	Service retention, inventory tracking, remarketing
OEMs	Monetising and distributing their vehicle data to enterprises

3. Industry Trends (2026 → 2030)

- **Data explosion:** McKinsey and S&P estimate up to ~25 GB/hour per connected car, and connected cars are projected to be ~95% of new vehicles sold by 2030. Most raw data must be filtered at the edge; only useful signal reaches the cloud.
- **Software-Defined Vehicles (SDV):** vehicles receive over-the-air updates and behave like rolling compute nodes, so data formats change more often.
- **Electrification:** mixed ICE/EV fleets need charging optimisation, battery state-of-health tracking and range prediction.
- **Cloud-agnostic pressure:** AWS IoT FleetWise closed to new customers in April 2026, a visible reminder of the risk of single-vendor lock-in.
- **Agentic AI:** AI copilots and agents are moving from dashboards to recommending and taking actions on fleet data.
- **Regulation:** UNECE R155/R156 require vehicle cybersecurity and software-update management for new vehicles in many markets; India's DPDP Rules were notified in November 2025; GDPR applies in Europe.

3.1 How Motorq Is Responding (From Public Sources)

- **Normalisation engine:** every OEM encodes events differently; Motorq continuously maps data from 25+ brands into one standard schema.
- **Streaming delivery:** customers consume data via portal, REST APIs and Kafka streams. In May 2026, Snowflake delivery went from about one hour to seconds (60x faster).

- **Motorq Fuse (April 2026):** an AI layer that monitors fleets, flags issues such as excessive idling, recommends actions and estimates the dollar impact of fixing them.
- **Engineering stack:** public job posts list Java, Go, Python and C#, distributed systems, AWS / Azure / GCP and agentic AI. Hiring rounds test DSA, OS, DBMS and REST API design.

4. The Problem at Scale

The core engineering problem is volume and velocity. The back-of-envelope estimate below uses conservative numbers for a single mid-sized platform.

Parameter	Value	Notes
Connected vehicles	100,000	One mid-sized fleet customer base
Events per vehicle	1 per second	Location, speed, battery, diagnostics
Ingest rate	~100,000 events/sec	Bursts of 3x at shift start or network recovery
Event size	~1 KB	JSON before compression
Raw volume	~8.6 TB/day, ~3 PB/year	Before compression and down-sampling

4.1 Why a Single SQL Database Fails Here

- **Write amplification:** every insert updates multiple B-tree indexes, so write throughput collapses as tables grow into billions of rows.
- **Lock and connection contention:** hundreds of thousands of concurrent writers exhaust connection pools and create lock waits.
- **Mixed workloads:** analytical scans over months of telemetry starve the transactional queries that dashboards and APIs depend on.
- **Vertical scaling limits:** a single primary node cannot scale writes horizontally without sharding.

Yet some data still needs strict ACID guarantees: fleet ownership, subscriptions, billing, access control and audit logs. A good solution uses the right store for each kind of data and explains the trade-offs.

5. Opportunity and Problem Spaces

Whoever turns this firehose into sub-second, trustworthy, actionable decisions owns the fleet-intelligence layer. The problem spaces below are starting points; you may choose one or define your own.

Problem Space	Who It Helps	Example Question to Answer
Predictive maintenance	Fleet managers	Which vehicles are likely to break down in the next 7 days?
EV charging and battery health	EV fleets, OEMs	When and where should each vehicle charge at lowest cost?
Driver safety scoring	Fleets, insurers	Which drivers show risky patterns, and what should change?
Asset recovery	Lenders	Where is this vehicle now, and is it behaving abnormally?
Fuel, idling and utilisation cost	Fleet finance teams	Where are we losing money, and how much can we save?
Multi-OEM data normalisation	Platform teams	How do we onboard a new OEM format without downtime?
Privacy-safe data sharing	OEMs, regulators	How do we share insights without exposing personal data?
Fleet carbon reporting	Sustainability teams	What are our emissions per route, and how do we cut them?

6. Your Challenge (Open-Ended)

Pick a real problem in the connected-vehicle space and build a working solution for it. There is no fixed specification: you choose the users, the features and the architecture. Depth beats breadth.

Minimum bar every solution must meet:

- Works on simulated data from at least 100,000 vehicles, generated by your own data simulator
- Processes events in real time and supports batch analytics over historical data
- Uses a combination of relational, NoSQL and (where useful) vector storage, each justified
- Exposes insights through secure APIs and a usable web interface
- Is containerised, tested and deployable on at least one cloud with no code changes needed for another

7. Technical Expectations

The table shows what reviewers look for in each area. Tools are examples only; choose any and justify your choices.

Area	What a Strong Solution Shows	Example Tools (Pick Any)
IoT ingestion	Realistic data generator (bursty, out-of-order, duplicate events); idempotent, schema-validated, back-pressure-aware intake	MQTT (EMQX, Mosquitto), Avro / Protobuf
Messaging	Durable, partitioned streams with replay	Kafka (recommended) or RabbitMQ
Big data pipeline	Real-time detection within seconds plus batch analytics over billions of rows	Kafka Streams, Flink, Spark on Hadoop / HDFS or S3, Parquet, Iceberg
Persistence	Polyglot storage with a reason for each store	PostgreSQL / TimescaleDB, Cassandra / MongoDB, Redis, Elasticsearch, Snowflake / ClickHouse
Full stack	Secure, paginated, rate-limited APIs and a clear UI	React; FastAPI (SQLModel), Spring Boot (Hibernate / JPA), ASP.NET Core, Go
ML and vector layer	A model that turns data into a decision, evaluated against a baseline	scikit-learn, PyTorch, pgvector, Qdrant
Agentic AI	An agent that answers or acts on fleet data, with guardrails and an audit trail	LangGraph, MCP tools, any LLM
Log analytics	Centralised logs, metrics and traces for a large event stream	Splunk or ELK, OpenTelemetry, Prometheus + Grafana
DevOps	Cloud-native microservices, IaC, CI/CD; cloud-agnostic deployment	Docker, Kubernetes / Helm, Terraform, GitHub Actions; AWS / GCP / Azure

8. Data Engineering and Database Design

- **Normalisation:** design the relational core (e.g. Fleet, Vehicle, Driver, Trip, Alert, Subscription) in Third Normal Form, and document any deliberate denormalisation.
- **Polyglot design:** combine relational, NoSQL and vector stores; state which data lives where and why.
- **Partitioning:** partition telemetry by time and vehicle; choose shard keys that avoid hot spots.
- **Data generation:** generate 100,000+ vehicles with realistic trips, faults and noise; the simulator itself is part of your solution.
- **SQL optimisation:** show EXPLAIN ANALYZE before and after for your slowest queries: composite and partial indexes, materialised views, keyset pagination, removing ORM N+1 queries.

Illustrative telemetry event (design your own):

```
{ "vin": "1HGCM82633A004352", "ts": "2026-09-25T10:15:02.120Z", "lat": 21.1702, "lon": 72.8311, "speed_kmh": 64.2, "soc_pct": 41, "odo_km": 18234.7, "dtc": ["P0301"], "evt": "HARSH_BRAKE", "seq": 88412 }
```

9. Algorithms and Data Structures

Use large-scale algorithms where your solution needs them. Examples:

Area	Example Application
Graph	Nearest reachable charger under remaining range (Dijkstra / A*); clusters of vehicles at unapproved depots (connected components)
Dynamic programming	Optimal EV charging schedule under time-of-use tariffs; segmenting noisy GPS into stops and trips
Regex and parsing	Validate VINs (17 characters, no I/O/Q, check digit); parse OBD-II fault codes such as P0301 from raw OEM payloads
Streaming algorithms	Sliding-window aggregates, de-duplication with Bloom filters, top-K with Count-Min Sketch, geohash indexing

10. System Design Expectations

- **CAP theorem:** choose and justify consistency or availability for each kind of data. For example, billing may need CP (strong consistency) while raw telemetry can be AP (eventual consistency). Discuss PACELC latency trade-offs.
- **Principles tested:** partitioning and sharding, replication, idempotency, exactly-once vs at-least-once delivery, back-pressure, circuit breakers, CQRS and event sourcing, cache invalidation, horizontal scaling, graceful degradation.
- **Data lifecycle:** a hot / warm / cold storage strategy with a retention and cost estimate.
- **Architecture style:** microservices with clear boundaries, layered or hexagonal internals, and documented decisions (ADRs).

11. Non-Functional Requirements

NFR	Target
Throughput	Sustain 100,000+ events/sec; survive a 3x burst for 5 minutes without data loss
Latency	Ingest to dashboard under 2 s; critical alert under 5 s; API p95 under 200 ms, p99 under 500 ms
Scalability	Stateless services scale horizontally; adding brokers or nodes needs no code change
Availability	No single point of failure; 99.9% target; recovers after a broker or pod is killed
Security	OAuth2 / OIDC + JWT, RBAC with tenant isolation, mTLS for devices, TLS 1.3 in transit, AES-256 at rest, secrets in a vault, OWASP Top 10 and API Top 10
Compliance	Audit logs for every data access and AI-agent action; masking of location data; retention and right-to-erasure flow (GDPR, DPDP)

12. Testing Expectations

Testing is heavily weighted. All suites should run automatically in CI on every push.

Test Type	Minimum Evidence
Unit	80%+ coverage on core services and algorithms (pytest, JUnit 5 + Mockito)
Integration and contract	Real broker, databases and cache via Testcontainers; contract tests between services (Pact)
Acceptance	BDD scenarios for key user stories (Cucumber, behave)
Performance and load	k6, Locust or Gatling at 100,000 events/sec; throughput, p95/p99, consumer lag; one soak test
Security	SAST (Semgrep, SonarQube), DAST (OWASP ZAP), dependency and image scans (Trivy)
Compliance and chaos	Verify audit trail and erasure; kill a pod or broker and prove recovery

13. Deliverables

- Completed Solution Document (use the provided template)
- Git repository with README, one-command local setup (docker compose) and a seeded 100K-vehicle dataset
- Architecture diagram, ER diagram (3NF) and 3–5 Architecture Decision Records
- Working end-to-end demo running on the simulated vehicle stream
- Test evidence: coverage report, load-test results, security scan reports, CI pipeline link

- DevOps pack: Dockerfiles, Helm charts or Kubernetes manifests, Terraform for at least one cloud, STRIDE threat model
- Algorithms and SQL write-up with complexity analysis and before/after query plans
- Demo video of 5 minutes maximum

14. Rules and Guidelines

- **Data:** use only synthetic or public data; no real personal or vehicle-owner data.
- **Open source and AI tools:** allowed, but must be declared in your Solution Document.
- **Code freeze:** only commits before the deadline are reviewed; tag the final commit v1.0-submission.
- **Originality:** your problem framing, design and code must be your team's own work.
- **Affiliation:** Motorq is used only as an industry reference. This is an independent academic exercise with no affiliation to Motorq.

15. Glossary

Term	Meaning
OEM	Original Equipment Manufacturer — the vehicle maker (e.g. Volvo, Subaru)
Telematics	Remote collection of vehicle data such as location, speed and diagnostics
OBD-II / DTC	On-board diagnostics standard / Diagnostic Trouble Code reported by the vehicle
VIN	17-character Vehicle Identification Number
MQTT	Lightweight publish–subscribe protocol designed for IoT devices
SoC / SoH	Battery State of Charge / State of Health
CAP / PACELC	Trade-offs between consistency, availability, partition tolerance and latency in distributed systems
3NF	Third Normal Form — a database design that removes redundancy and update anomalies
ADR	Architecture Decision Record — a short note of a design decision and its consequences